

An optimized Q-Learning algorithm for mobile robot local path planning

Qian Zhou^{a,*}, Yang Lian^b, Jiayang Wu^a, Mengyue Zhu^a, Haiyong Wang^b, Jinli Cao^c

^a School of Modern Posts, Nanjing University of Posts and Telecommunications, Nanjing, 210003, Jiangsu, China

^b School of Computer Science, School of Software and School of Cyberspace Security, Nanjing University of Posts and Telecommunications, Nanjing, 210037, Jiangsu, China

^c Department of Computer Science and Computer Engineering, La Trobe University, Melbourne, Australia

ARTICLE INFO

Keywords:

Mobile robot
Q-Learning algorithm
Local path planning
Reinforcement learning
Adaptive learning rate

ABSTRACT

The Q-Learning algorithm is a reinforcement learning technique widely used in various fields such as path planning, intelligent transportation, penetration testing, among others. It primarily involves the interaction between an agent and its environment, enabling the agent to learn an optimal strategy that maximizes cumulative rewards. Most non-agent-based path planning algorithms face challenges in exploring completely unknown environments effectively, lacking efficient perception in unfamiliar settings. Additionally, many Q-Learning-based path planning algorithms suffer from slow convergence and susceptibility to getting stuck in local optimal solutions. To address these issues, an optimized version of the Q-Learning algorithm (Optimized Q-Learning, O-QL) is proposed and applied to local path planning of mobile robots. O-QL introduces novel Q-table initialization methods, incorporates a new action-selection policy, and a new reward function, and adapts the Root Mean Square Propagation (RMSprop) method in the learning rate adjustment. This adjustment dynamically tunes the learning rate based on gradient changes to accelerate learning and enhance path planning efficiency. Simulation experiments are carried out in three maze environments with different complexity levels, and the performance of the algorithm in local path planning is evaluated using steps, exploration reward, learning rate change and running time. The experimental results demonstrate that O-QL exhibits improvements across all four metrics compared to existing algorithms.

1. Introduction

Path planning refers to finding an appropriate motion path in the environment with obstacles, so that the subject can safely bypass all obstacles from the given starting point and reach the end point without collision [1]. It can be divided into global path planning and local path planning [2–4]. Current research on path planning involves many fields, such as mobile robots [5], drones [6], automatic driving [7], transportation [8], etc. There are abundant researches on path planning of mobile robots, and many mature algorithms for path planning of known environments have been designed [9]. In the drones field, it is often applied to scenes with complex environments, aiming to plan a safe and fast path quickly and accurately [10]. In the field of intelligent transportation, autonomous driving technology is closely linked to path planning, with particular attention to its application on highways [11]. At the same time, attribute reduction [12] can be applied to simplify the complexity of path planning, thereby improving the efficiency of path search and alleviating traffic pressure [13]. It can also be combined with homomorphic encryption based on

greedy algorithm to realize real-time path planning under encryption [14,15]. However, most of the existing path planning algorithms are not effective in exploring completely unknown environments and cannot achieve effective perception of unfamiliar environments [16].

Machine learning (ML) is commonly used for path planning of robots [17]. In early studies, supervised learning is a good choice for robots to manage movement on fixed maps. However, when the map changes, the robot needs to be fully trained using the old and new sensory data-motion pairs [18]. Reinforcement learning (RL) also belongs to machine learning and can be used to understand and automate goal-oriented learning and decision making [19]. Unlike supervised learning, RL agents do not use examples of correct or incorrect actions to facilitate learning, but instead receive positive and negative rewards for their chosen actions [20]. Q-Learning (QL) algorithm is a classic algorithm of reinforcement learning, which has proved its effectiveness and wide application in many fields in recent years [21], the most common one is the path planning of specific agents, and now it is also applied in the field of security. For example, combining penetration

* Corresponding author.

E-mail addresses: zhouqian@njupt.edu.cn (Q. Zhou), 1321048425@njupt.edu.cn (Y. Lian), 1223096926@njupt.edu.cn (J. Wu), 1223096927@njupt.edu.cn (M. Zhu), why@njupt.edu.cn (H. Wang), J.Cao@latrobe.edu.au (J. Cao).

<https://doi.org/10.1016/j.knosys.2024.111400>

Received 8 December 2023; Received in revised form 8 January 2024; Accepted 11 January 2024

Available online 15 January 2024

0950-7051/© 2024 Elsevier B.V. All rights reserved.

testing to find the attack path of attack graph [22], using neural networks to fit the mapping relationship between state space and action space, and building an intelligent network penetration testing model based on deep reinforcement learning [23]. However, it is worrying that the traditional Q-Learning algorithm sets the Q value as equivalent or random value in the initialization process, that is, it learns in the environment without prior knowledge, which will lead to slow convergence of the algorithm [24].

Therefore, scholars have launched a wide range of attempts to speed up the convergence of the algorithm. Zhang et al. [25] added the reward value of diagonal motion to the reward function, so that the agent can reduce blind search during exploration, improve the learning efficiency of the algorithm and shorten the moving path of the agent. However, it may cause the agent to prefer the diagonal path and ignore other better paths. Gao et al. [26] proposed an improved Q-learning algorithm, adding a learning layer on the base of Q-Learning algorithm, and applied it to path planning of mobile robots in multi-obstacle environment. This improved method accelerates the learning process algorithm by adding learning layers, and shows the generalization ability under multi-obstacle environment. However, this improvement also brings problems such as increased algorithmic complexity, possible introduction of new hyper-parameters, and increased memory consumption. Yuan et al. [20] proposed a new multi-step approach, which, unlike traditional Q-Learning algorithm, uses a new return function that changes the discount of future rewards while reducing immediate rewards when selecting current state actions, aiming to improve the data efficiency of enhancing deep reinforcement learning. However, it augments the difficulty of tuning the algorithm, and may also lead to the increase in the sensitivity of the algorithm to hyper-parameters. Song et al. [27] integrated genetic algorithm and Q-Learning algorithm, so that each exploration of the algorithm is no longer limited to one step, and smooth the key nodes of the optimal path by using Bessel curve, so that the algorithm reduces the probability of falling into the local optimal and can better realize the path planning under the dynamic map. However, there are some disadvantages such as complex parameter adjustment and limited robustness to dynamic environment. Meerza et al. [28] proposed a fusion path planning algorithm that combined Q-Learning algorithm and Particle swarm optimization (PSO), using PSO to improve the iteration of Q-table and increase the speed of the algorithm. However, the computational complexity and the probability of falling into the local optimal solution are increased in this method. Bai et al. [29] proposed a Q-Learning algorithm that used Flower Pollination Algorithm (FPA) to initialize Q-table. Before executing Q-Learning algorithm, FPA algorithm could explore the environment, then initialize Q-table and assign Q values. This will allow agents to use prior knowledge to explore environment when doing path planning, rather than starting with zero information. But the accuracy of initializing the Q-table may influence the efficiency and convergence of the algorithm [30].

The contributions of this study are summarized as follows:

1. A novel Q-learning initialization method has been proposed, including Q-table initialization, action-selection policy, and a reward function, so that the agent can move towards the endpoint more targetedly, reduce the blindness of initial exploration, and accelerate the agent's perception of the environment;
2. A Root Mean Square Propagation based learning rate adaptive adjustment method has been proposed, which allows the agent to adaptively adjust the learning rate based on historical gradient information, avoiding the problem of excessive or insufficient learning rate and accelerating the convergence speed of exploration;
3. Experiments show that the algorithm's performance in local path planning using metrics such as step counts, exploration rewards, learning rate variations, and runtime have been improved compared with existing algorithms.

Table 1

Symbol meaning reference table.

Symbol	Meaning
α	Learning rate
γ	Discount factor
ϵ	Greed factor
S, s_t	State set, State at time t
A, a_t	Action set, Action at time t
O	Obstacle grille collection
E	Barrier free grille collection
β	The decay coefficient for gradient moving average
λ	Small constant
Q	Q-table value
r	Reward value
x	Relative x-coordinate
y	Relative y-coordinate

The remainder of this paper is organized as follows: Section 2 models the mobile robot path planning problem based on the Q-Learning algorithm. Section 3 introduces the optimized Q-Learning algorithm. Section 4 conducts comparative experiments and analyzes the results. Finally, some conclusions are drawn in Section 5.

2. Modeling the path planning problem of mobile robots based on Q-Learning

2.1. Q-Learning algorithm

The Q-Learning algorithm is one of the popular algorithms in reinforcement learning, and it is a model-free Temporal Difference (TD) algorithm [31]. Its purpose is for an intelligent agent to learn an optimal policy that maximizes the cumulative reward through interaction with the environment. The main symbols and their meanings involved in the Q-Learning algorithm are shown in Table 1.

The learning process of Q-Learning adopts the form of a Markov Decision Process (MDP), which can be represented by a five-tuple (S, A, P, R, γ) , where S is a finite set of states; A is a finite set of actions; P is the state transition probability matrix based on actions; R is the reward function based on states and actions; and γ is a discount factor.

The Q-Learning algorithm uses the Q-value function to predict the long-term return of taking a specific action in a particular state. The Q-values are stored in the Q-table. In the environment, the intelligent agent has several executable actions in each state, and executing these actions will lead the agent to a new state, thus updating the Q-table. Assuming there are m states and n actions, the size of the Q-table is $m \times n$.

The algorithm finds the optimal action that the intelligent agent should take in different states by querying the maximum Q-value in each row of the Q-table. The Q-value $Q(s_t, a_t)$ represents the estimated value of the reward obtained when the agent takes action a_t in state s_t at that time. The Q-value update formula is as shown in Eq. (1) [21]:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha [R(s_t, a_t) + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \quad (1)$$

Where s_t represents the state of the intelligent agent at time t , a_t represents the action taken by the agent in state s_t at this moment, $Q(s_t, a_t)$ represents the current Q-value; $R(s_t, a_t)$ represents the reward obtained by the agent when taking action a_t in state s_t ; γ represents the discount factor; $Q(s_{t+1}, a_{t+1})$ represents the Q-value in the new state s_{t+1} and action a_{t+1} , and $\max_{a_{t+1}} Q(s_{t+1}, a_{t+1})$ represents the maximum potential future reward that the agent may obtain when taking action a_t and reaching state s_{t+1} .

It should be noted that α is the learning rate, $\alpha \in [0, 1]$, which indicates the importance of the experience learned by the agent at the new Q-value. When $\alpha = 0$, it means the agent does not learn any

information from the new Q-value and only values the old information; when $\alpha = 1$, it means the agent does not care about the old experience and only focuses on the newly learned information.

If the expression $-Q(s_t, a_t)$ in the brackets of α is extracted and combined with $Q(s_t, a_t)$, the Q-value update formula can be represented in the form of Eq. (2):

$$Q(s_t, a_t) = (1 - \alpha)Q(s_t, a_t) + \alpha[R(s_t, a_t) + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})] \quad (2)$$

Where $(1 - \alpha)Q(s_t, a_t)$ represents the proportion of the old Q-value in the new Q-value, and $\alpha[R(s_t, a_t) + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})]$ represents the reward learned by the agent from this action.

On the other hand, the traditional Q-Learning algorithm uses the ϵ -greedy strategy as the basis for action selection. During execution, the algorithm generates a random number p between $[0, 1]$. If p is less than the greedy factor ϵ , the intelligent agent chooses the optimal action (i.e., the action with the largest Q-value in that state); otherwise, the agent randomly selects an action from all executable actions in that state. The greedy factor ϵ is generally set to a value close to 1 (e.g., 0.9) to ensure that the algorithm has a high probability of choosing the action with the highest Q-value, while still retaining some randomness to avoid getting stuck in a situation where it continuously chooses the optimal action without exploring the environment further.

The specific implementation steps of the Q-Learning algorithm can be summarized as follows:

1. Initialize the Q-table;
2. Determine the environment and the state of the intelligent agent;
3. Check if the agent is in the final state. If not, select an action based on the policy, execute the action to reach the next state, and receive a reward [32];
4. Update the Q-table based on Eq. (1);
5. Update the state of the intelligent agent and repeat steps 3 and 4 until it reaches the final state.

2.2. Problem description

The Q-Learning-based mobile robot path planning aims to find the optimal sequence of actions for the robot to move from the starting point to the endpoint, in order to achieve the shortest and collision-free path. The solution approach of Q-Learning maps the mobile robot to an intelligent agent. In an unknown environment (O, E) , the intelligent agent learns and updates the Q-table through trial and error, i.e., selecting different actions at different states st and learning from the feedback results. This learning process enables the intelligent agent to gradually find the optimal strategy to reach the target state [33]. Through continuous iteration, the Q-Learning algorithm can converge to an optimal policy, allowing the mobile robot to effectively plan the shortest and collision-free path in an unknown environment.

2.3. Environment modeling

Gridification of the environment space is one of the commonly used methods in solving mobile robot path planning problems based on Q-Learning [34]. Gridification refers to dividing the continuous environment space into discrete grids, which enables the discrete representation of the robot's position, orientation, and other states [35].

As a result, the environment in which the mobile robot operates can be abstracted as a tuple (O, E) , where O represents the set of obstacle grids ($o \in O$), and E represents the set of empty space grids ($e \in E$). Treating the mobile robot as a point mass, any given current or next action of the robot can uniquely determine whether the position belongs to the set of obstacle grids or empty space grids, as shown in Eq. (3):

$$L(s_t, a_t) = l_t, l_t \in \begin{cases} E, l_t = e \\ O, l_t = o \end{cases} \quad (3)$$

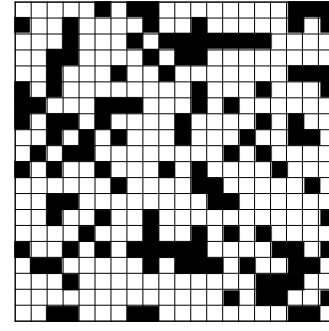


Fig. 1. Modeling the environment with grid.

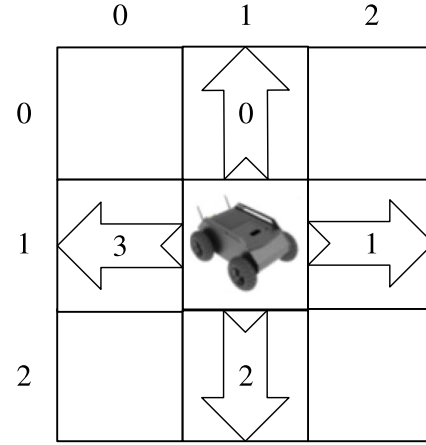


Fig. 2. Discrete action set.

Gridification enables the discretization of states, facilitating the construction of the Q-table where each entry corresponds to a state-action pair, making it easier to implement and learn the algorithm. Additionally, gridification reduces the size of the state space. In actual environments, mobile robots may face an infinite number of states. Without discretizing the states, the Q-table would become extremely large, making it difficult for the algorithm to converge and consume more computational resources as shown in Fig. 1.

2.4. Discrete action set

Since the scale of the gridified grids is much smaller than the turning radius of the mobile robot, we assume that the mobile robot has the ability to move freely in the four surrounding grids. Therefore, we have designed four basic discrete actions for the mobile robot, namely moving up, moving right, moving down, and moving left, each by 1 unit length, numbered from 0 to 4 as shown in Fig. 2.

3. Optimized Q-Learning algorithm

3.1. Optimization ideas

In solving the mobile robot path planning problem based on Q-Learning, this paper adopts multiple optimization methods and proposes the Optimized Q-Learning algorithm (O-QL). Firstly, through the Q-table initialization method based on prior knowledge, the Euclidean distance between the starting point and the endpoint is utilized to guide the initial learning of the intelligent agent. Secondly, addressing the exploration-exploitation dilemma, a novel action selection strategy combining ϵ -greedy and Boltzmann strategies are employed to promote

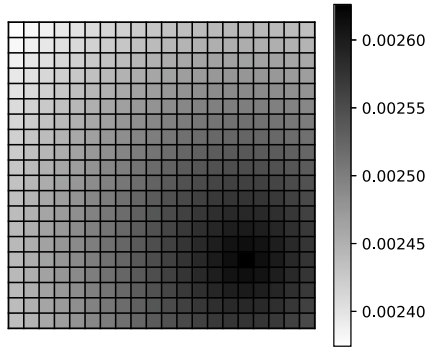


Fig. 3. Optimization of Q-table initialization.

the exploratory behavior of the intelligent agent during the initial training phase. Additionally, a reward function is designed to accelerate the convergence speed of Q-Learning by avoiding reward sparsity through a continuous reward function. Finally, the RMSprop dynamic learning rate adjustment optimization method is introduced to dynamically adjust the learning rate based on gradient information, better adapting to environmental changes. These comprehensive optimization measures aim to improve the efficiency and performance of the intelligent agent's path planning.

3.2. Specific optimization methods

3.2.1. Q-table initialization optimization

During the initial learning phase, the intelligent agent has very little knowledge of the environmental states and blindly selects actions from the action set. Therefore, using prior knowledge to initialize the Q-table is an effective way to reduce the convergence time of the Q-Learning algorithm [36]. In O-QL, the Q-table of the agent is initialized using the Euclidean distance between the endpoint and the current position of the mobile robot. Assuming a grid cell is represented as (x_i, y_j) , and the grid cell of the endpoint is represented as (x_d, y_d) , the Q-table value of this grid cell is initially calculated in Eq. (4):

$$Q([x_i, y_j]) = \frac{1}{\exp(\sqrt{(x_d - x_i)^2 + (y_d - y_j)^2})} \quad (4)$$

Specifically, assuming a 20*20 maze environment, regardless of the distribution of obstacles and the starting coordinates, assuming the endpoint coordinates (x_d, y_d) are (15,15), the above formula can uniquely determine the initialized Q-value for each grid cell (x_i, y_j) as shown in Fig. 3.

In this scenario, the Q-table is a 20*20*4 two-dimensional matrix (environment length * environment width * number of actions). If the agent's next action from the current state would lead it to a particular grid, the corresponding Q-value is updated to the value calculated through Eq. (4). If the next action from the current state would cause the agent to exit the environment's bounds, the corresponding Q-value is set to 0 [18]. This initialization method ensures that each value in the Q-table is uniquely determined, aiding the agent in purposefully moving towards endpoints because the agent tends to select the action with the highest Q-value for each state among the four available actions. Simultaneously, this method also exhibits some level of generalization as the agent retains some action-selection capability, preventing it from solely relying on the initial Q-table movement and thereby avoiding getting stuck in local optimal solutions without exploring new paths.

The randomness in learning can be greatly reduced by using this initialization method, allowing the intelligent agent to move more purposefully towards the endpoint.

3.2.2. Action-selection policy optimization

Reinforcement learning faces exploration–exploitation dilemma, thus the performance of the Q-Learning algorithm is closely related to the chosen exploration strategy. Two of the most common exploration strategies are the ϵ -greedy strategy and the Boltzmann strategy [37].

Our O-QL employs a new action selection strategy that combines the ϵ -greedy and Boltzmann strategies, which has the advantage of avoiding local optima and accelerating convergence speed [38]. In the initial training phase, this strategy makes the intelligent agent prefer to explore unknown environments. When the number of training episodes reaches a certain threshold, this strategy makes the intelligent agent inclined to utilize the available prior information.

Initially, the intelligent agent has no knowledge of the environmental states and must randomly select actions from the action set. Due to this randomness, it is possible that executing an action results in the state being the same as the previous episode, leading to unnecessary repetition of states. The combined ϵ -greedy and Boltzmann strategy is described in Algorithm 1:

Algorithm 1 Action-selection Policy Optimization Algorithm

Require: ϵ -greedy parameter ϵ , training threshold $split_episode$, action set A

Ensure: Next action a

```

1: Randomly generate  $p \in (0, 1)$ 
2: if  $p > \epsilon$  then
3:   if  $episode \geq split\_episode$  then
4:      $a \leftarrow \text{Boltzmann}(A)$ 
5:   else
6:      $a \leftarrow \epsilon\text{-greedy}(A)$ 
7:   end if
8: else
9:   while  $s' == s$  do
10:     $a \leftarrow \text{Random}(A)$ 
11:   end while
12: end if
13: return  $a$ 

```

3.2.3. Reward function optimization

In O-QL, the goal of the intelligent agent is to obtain the maximum cumulative reward through interaction with the environment (from the initial state to the target state). In this study, the reward function is designed as shown in Eq. (5):

$$R = \begin{cases} -10, R = motion1 \\ 40, R = motion2 \\ -5, R = motion3 \\ r, R = motion4 \end{cases} \quad (5)$$

Where *motion1* represents the intelligent agent colliding with obstacles or going beyond the environment boundaries, with a reward value of -10 ; *motion2* represents the intelligent agent reaching the endpoint, with a reward value of 40 ; *motion3* represents the intelligent agent visiting a node that has been visited in the current round, with a reward value of -5 ; *motion4* represents other situations, with the specific value determined by the continuous reward function r .

We determined this set of reward function values through an integer search, optimizing the performance of O-QL. Setting *motion1* and *motion3* as penalties is reasonable as they have a negative impact on the completion of the final task. However, the penalty for *motion3* is slightly lower, considering scenarios where the agent might backtrack due to encountering dead ends. *motion2* represents a strong positive incentive, signifying the completion of the final task, hence receiving the highest reward value.

To avoid excessively sparse reward functions and consequently accelerate the convergence speed of the Q-learning algorithm, O-QL uses the continuous reward function [39]. The expression for the continuous

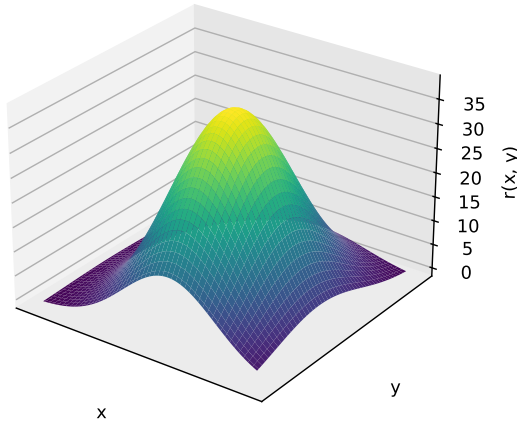


Fig. 4. Continuous reward function.

reward function r corresponding to the relative coordinate points in the environment is defined in Eq. (6):

$$r = r_0 + r_{\max} \times \exp\left(-\left(\frac{x_i - x_d}{G_x}\right)^2 - \left(\frac{y_i - y_d}{G_y}\right)^2\right) \quad (6)$$

Where r_0 represents the basic reward value, which is -1 when the intelligent agent moves one step to empty space (regardless of whether it moves left, right, up, or down), and $r_{\max}$ represents the reward value to be reached at the endpoint. G_x and G_y respectively control the decrease rate of the reward function along the x -axis and y -axis. To put it more precise, G_x is set to 1.5 and G_y to 2.0. The plot of the r function is shown in Fig. 4.

The main reasons why using continuous reward functions leads to better performance are as follows [40]: Continuous reward functions can provide more information by offering different reward values at each time step, rather than providing a single discrete reward only upon task completion. This continuity allows the agent to better understand the effects of its actions before reaching the goal, aiding in fine-tuning its strategy. Additionally, continuous reward functions tend to be smooth mathematical functions. In contrast, discrete reward functions may exhibit abrupt transitions between different states or actions, making smooth reward functions more helpful for the agent to learn the environment's continuity and patterns of change, thus better inferring reward values for unknown states. Moreover, Gaussian functions, as one form of continuous and smooth mathematical functions, possess favorable mathematical properties. They provide a smooth reward distribution, aiding the agent in comprehending gradual changes between states or actions. Hence, in O-QL, the continuous reward function in Eq. (6) is a Gaussian function.

3.2.4. Dynamic learning rate adjustment optimization

In the Q-learning algorithm, dynamic learning rate adjustment is a key aspect of improving the algorithm's performance. In traditional Q-learning algorithms, the learning rate is usually fixed, but in practical problems, the dynamic nature of the environment may lead to the inadequacy of the learning rate. To address this issue, this paper introduces a dynamic learning rate adjustment optimization method, namely Root Mean Square Propagation (RMSprop) [41].

RMSprop adjusts the learning rate by maintaining a moving average of the squared gradient, thereby adapting to changes in the environment. Its core idea is to consider the historical information of the gradient, allowing for different learning rates for different parameters during the learning process [42]. Specifically, for learning rate α , the update rules of RMSprop are Eq. (7) and Eq. (8):

$$E[g^2]_t = 0.9E[g^2]_{t-1} + 0.1g_t^2 \quad (7)$$

$$\alpha_{t+1} = \alpha_t - \frac{\alpha_0}{\sqrt{E[g^2]_t + \lambda}} g_t \quad (8)$$

Where $E[g^2]_t$ is the moving average of the squared gradient, g is the current gradient, α_0 is the initial learning rate, and λ is a small constant to prevent division by zero. To apply RMSprop in Q-learning, we make corresponding adjustments to the update of the Q-table. Specifically, the update rule for the Q-table for state s_t and action a_t is as Eq. (9):

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha_t \frac{1}{\sqrt{E[\nabla Q(s_t, a_t)^2] + \lambda}} \delta \quad (9)$$

Where δ is the Temporal Difference Error (TD error), and $E[\nabla Q(s_t, a_t)^2]$ is the moving average of the squared gradient. The specific algorithm flow is shown in Algorithm 2.

Algorithm 2 Dynamic Learning Rate Adjustment Algorithm

Require: Initial Q value, learning rate α , decay coefficient for the moving average of squared gradients β , small constant λ

Ensure: Optimized Q value

- 1: Initialize Q -values randomly or with prior knowledge
- 2: Initialize $E[\nabla Q^2]$ to a small positive value
- 3: **for** each episode **do**
- 4: **for** each step in the episode **do**
- 5: Choose an action a using the combined ϵ -greedy and Boltzmann strategy
- 6: Take action a , observe the next state s' and the reward r
- 7: Calculate the TD error δ
- 8: Update $E[\nabla Q^2]$ using the decay factor β
- 9: Update Q -values using the RMSprop update rule
- 10: Move to the next state s
- 11: **end for**
- 12: **end for**
- 13: **return** Q -values

4. Experimentation and analysis

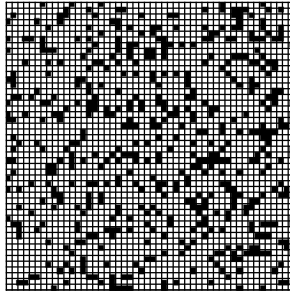
4.1. Experimental environment and setup

To validate the algorithm's feasibility and generalization ability, we tested its performance in a randomized obstacle grid environment sized 50×50 . We generated a total of 3 sets of grid environments, where the obstacle grids in each set accounted for 20%, 30%, and 40% of the total grid cells, respectively. The three grid environments are illustrated in Fig. 5.

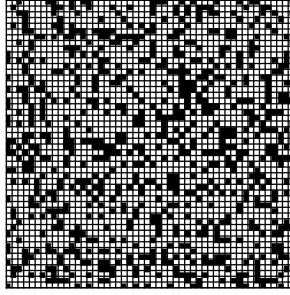
Assuming the obstacles' positions remain fixed after generation, the objective for the agent is to find the shortest and least collision path from $(0, 0)$ to $(49, 49)$. The robot can only move one grid cell at a time in four directions: up, down, left, or right. There must be at least one path from the starting point to the destination.

We considered the characteristics and advantages of two algorithms: Genetic Algorithm Q-Learning (GA-QL) [26] and Expected n-step DQN (EnDQN) [19]. GA-QL, based on genetic algorithms, can search for optimal solutions in the search space, demonstrating a certain level of efficiency and robustness in reinforcement learning. On the other hand, EnDQN combines n-step expected return estimation with deep Q networks, enhancing learning effectiveness by reducing sampling variance and improving convergence speed. Both algorithms have shown certain performance and potential in reinforcement learning tasks. Thus, we chose to compare them with the proposed Optimized Q-Learning (O-QL) in this paper to comprehensively evaluate the performance and advantages/disadvantages of different algorithms in problem-solving. The metrics for comparison are:

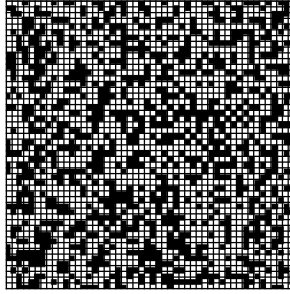
1. **Step:** The number of steps represents the times the agent explores the environment from the starting point to reaching the endpoint within one episode. Variations in step counts across episodes reflect algorithm performance. High step counts indicate the agent getting lost in the environment, while convergence to a specific value implies the agent has found a shortest path.



(a) 20% Obstacle Density



(b) 30% Obstacle Density



(c) 40% Obstacle Density

Fig. 5. Grid-based maps used in the experiment.

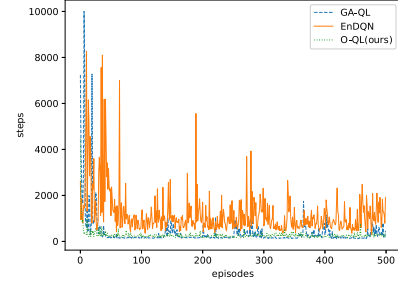
2. **Reward:** Reward is the sum of reward values obtained by the agent while traversing each grid cell along the path within one episode. Changes in reward across episodes depict algorithm performance. Low reward values suggest multiple collisions or extensive exploration before reaching the endpoint, while convergence in reward values implies step count convergence.
3. **Learning rate:** Learning rate indicates the significance of the experiences learned by the agent during an exploration round in updating new Q-values. Variations in learning rate across episodes demonstrate the agent's adaptive capabilities.
4. **Optimization time:** Optimization time refers to the duration taken by the agent to converge to an optimal path in an unknown environment. It reflects the algorithm's efficiency.

The experiments were conducted on a Windows 10 64-bit operating system (CPU: Intel Core i7-6500U, Memory: 16 GB) using Python 3.11 environment. The main parameter settings for the experiment are shown in Table 2.

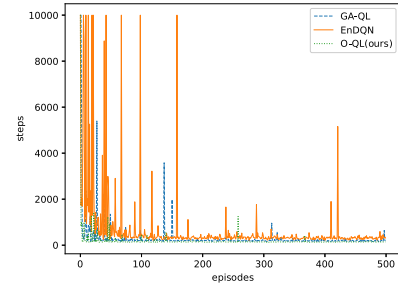
Table 2

Main parameters.

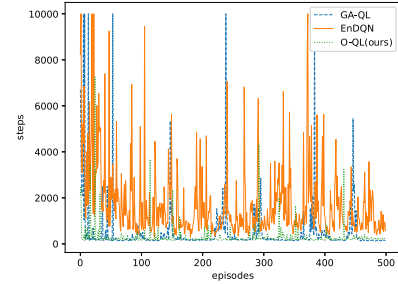
Parameter	Value
Iteration count <i>episodes</i>	500
Initial learning rate α	0.001
Training threshold <i>split_episode</i>	$0.5 \times \text{episodes}$
Discount factor γ	0.9
Greedy factor ϵ	0.2



(a) 20% Obstacle Density



(b) 30% Obstacle Density



(c) 40% Obstacle Density

Fig. 6. Steps variation with episode.

4.2. Result analysis

4.2.1. Comparison of step

This section explores the variation of step counts across episodes within the predefined three sets of grid environments. Experimental results conducted in environments with obstacle proportions of 20%, 30%, and 40% are illustrated in Fig. 6.

The horizontal axis of the graph represents the number of episodes, while the vertical axis represents the number of steps attempted by the agent in a single episode. The three lines correspond to three Q-Learning-based path planning algorithms. From the experimental results, it is evident that all three algorithms exhibit an initial fluctuation followed by a decrease and eventual stabilization in step counts across episodes. The initial fluctuation arises from the agent's limited

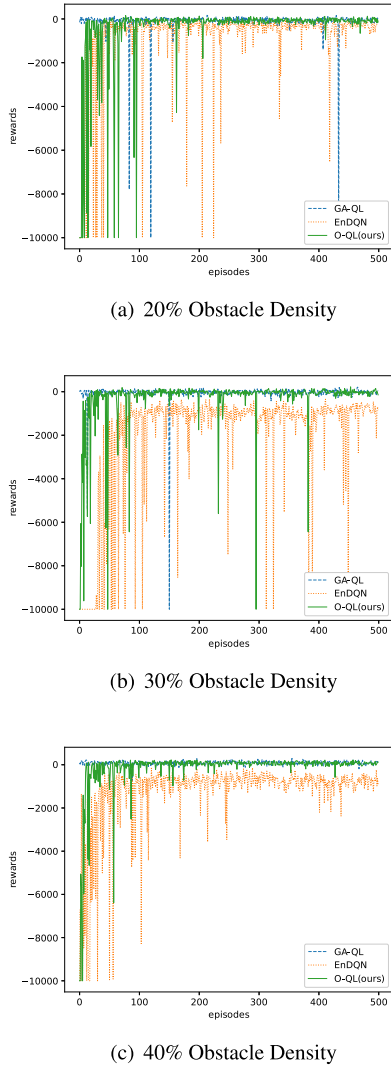


Fig. 7. Reward variation with *episode*.

understanding of the environment in the early stages, leading to relatively blind action selection and consequently more exploratory steps. As the number of successful arrivals at the endpoint increases, if the agent accurately learns environmental information, it can more swiftly identify the shortest path to the endpoint.

Our O-QL algorithm demonstrates the fastest convergence speed in step counts and the lowest convergence value. This is attributed to O-QL considering the positional relationship between corresponding grids and the endpoint during Q-table initialization. As a result, the agent initially prioritizes grids closer to the endpoint, reducing blind exploration. While GA-QL converges similarly to O-QL in step counts, it experiences numerous early unsuccessful attempts due to a lack of targeted Q-table initialization. EnDQN, with its challenging parameter tuning and weaker generalization ability, exhibits lower adaptability for the same initial parameters, struggling to learn the complex features of the grid environment and thus getting stuck in local optima.

4.2.2. Comparison of reward

This section investigates the variation of rewards across episodes within the predefined three sets of grid environments. The experimental results conducted in environments with obstacle proportions of 20%, 30%, and 40% are illustrated in Fig. 7.

The horizontal axis of the graph represents the number of episodes, while the vertical axis represents the reward values obtained by the

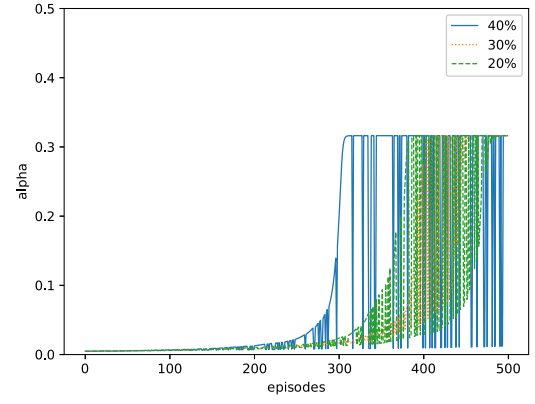


Fig. 8. Learning rate variation with *episode*.

agent in a single episode. The three lines correspond to three different algorithms. From the experimental results, it is observed that in the early exploration phase, the reward values per episode are negative and at a significantly low level. However, as the agent's step counts begin to converge, the rewards also converge to a comparatively higher level, indicating that the agent perceives it has found the shortest path.

Our O-QL algorithm also demonstrates the fastest convergence speed and the highest convergence value in rewards. This is attributed to O-QL's utilization of a continuous reward function to expedite convergence, prompting the agent to move towards the endpoint while avoiding revisiting grids from the current episode and minimizing collision-induced negative rewards. GA-QL, using a discrete reward function, leads to unguided agent movements and struggles to learn environmental features. EnDQN, despite having a continuous reward function, suffers from slow updates when confronted with complex environment maps.

4.2.3. Comparison of learning rate

This section explores the variation of the learning rate across episodes within the predefined three sets of grid environments. The experimental results conducted in environments with obstacle proportions of 20%, 30%, and 40% are illustrated in Fig. 8.

The horizontal axis of the graph represents the number of episodes, while the vertical axis represents the learning rate utilized by the agent when updating the Q-table in a single episode. Different curves correspond to varying obstacle densities. O-QL adopts RMSprop for dynamic adaptive learning rate adjustments, while GA-QL and EnDQN use constant learning rates initially set at 0.001. As episodes progress, O-QL adjusts its learning rate considering the historical information of gradients, adapting to the changing circumstances and unfamiliar environments.

Across scenarios with three different obstacle densities, the trend in learning rate variation is consistent: stabilizing around 0.001 for the initial 200 episodes, followed by a gradual increase characterized by fluctuations. This pattern emerges due to the relatively simple and limited state transitions encountered by the agent in the initial phase, resulting in consistent gradient information that does not prompt significant fluctuations in the learning rate. However, as the training progresses, the agent explores more complex and diverse state transitions, intensifying gradient variations, thereby influencing the adjustment of the learning rate. Furthermore, changes in the reward signals also contribute to fluctuations in the learning rate. With accumulating experience, a wider array of reward signal types is encountered. Specific reward signals associated with particular states can prompt rapid gradient changes, affecting the adaptation of the learning rate.

The learning rate determines how much experience the agent can draw from its current actions for the next iteration. Therefore, dynamically adaptive learning rates enable the agent to absorb diverse

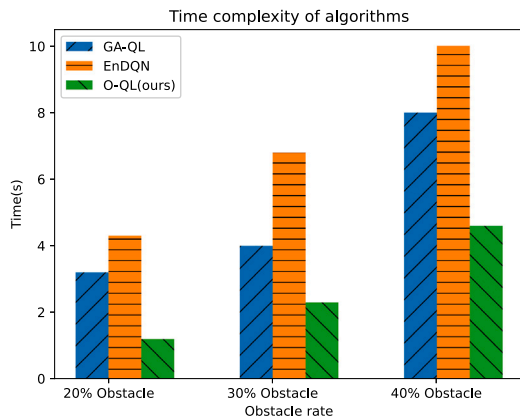


Fig. 9. Time comparison for optimization search.

experiences from different action attempts, facilitating faster adaptation to unfamiliar environments. This accelerates the convergence process and helps avoid getting trapped in local optima.

4.2.4. Comparison of optimization time

This experimentation demonstrates the optimization time, offering an intuitive measure of an algorithm's temporal performance. The experimental results conducted in environments with obstacle proportions of 20%, 30%, and 40% are depicted in Fig. 9.

The horizontal axis represents the proportion of obstacle grids to the total grids, while the vertical axis indicates the optimization time for the respective algorithm. From the experimental results, it is evident that our O-QL algorithm exhibits the best time efficiency. This is attributed to its lightweight methodology, involving simple distance and gradient computations, which simplifies the computational aspect. Additionally, its fast convergence reduces ineffective exploration attempts, contributing to its excellent time performance. GA-QL, due to its integration of genetic algorithms, incurs costs in computing optimal populations with each update, while EnDQN, involving numerous parameters, leads to significantly higher computational costs per iteration compared to the simpler gradient calculations of O-QL.

5. Conclusion

Our research proposes improvements to the Q-Learning algorithm by introducing new reward functions and dynamically adjusting the learning rate. These enhancements aim to address issues like slow convergence and low efficiency in local path planning encountered by traditional algorithms. The algorithm demonstrates significantly improved convergence speed while also strengthening stability and adaptability. Experimental results indicate that the enhanced algorithm outperforms traditional Q-Learning algorithms and other improved Q-Learning algorithms in convergence speed, loss, rewards, and operational efficiency.

In future research, a further integration of technologies such as deep learning is suggested for enhancing reinforcement learning algorithms. This integration aims to tackle more complex and real-world path planning problems and apply them to other domains, such as seeking attack paths in attack graphs. The study also aims to stay abreast of advancements made by other researchers in this field to collectively drive the development and practical applications of reinforcement learning algorithms.

CRedit authorship contribution statement

Qian Zhou: Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Conceptualization. **Yang Lian:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis. **Jiayang Wu:** Visualization, Software, Resources. **Mengyue Zhu:** Visualization, Validation. **Haiyong Wang:** Supervision, Data curation. **Jinli Cao:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work is supported by the National Natural Science Foundation of China under the grant Nos. 62372244 and 61902199.

References

- [1] H.D. Zhang, R. Zheng, Y.M. Cen, Present situation and future development of mobile robot path planning technology, *J. Syst. Simul.* 17 (2) (2005) 439–443.
- [2] R.L. Ma, Z.X. Guan, Summarization for present situation and future development of path planning technology, *Mod. Mach.* 3 (2008) 22–24.
- [3] Y.Z. Cong, S.G. Ponnambalam, Mobile robot path planning using ant colony optimization, in: 2009 IEEE/ASME International Conference on Advanced Intelligent Mechatronics, 2009, pp. 851–856.
- [4] X. Wang, Q.X. Zhu, Y.H. Zhu, Review of path planning algorithms for mobile robots, *Comput. Eng. Appl.* 59 (20) (2023) 51–66.
- [5] H. Zhang, W. Lin, A. Chen, Path planning for the mobile robot: A review, *Symmetry* 10 (10) (2018) 450.
- [6] Y. Wu, K.H. Low, B. Pang, et al., Swarm-based 4D path planning for drone operations in urban environments, *IEEE Trans. Veh. Technol.* 70 (8) (2021) 7464–7479.
- [7] X. Hu, L. Chen, B. Tang, et al., Dynamic path planning for autonomous driving on various roads with avoidance of static and moving obstacles, *Mech. Syst. Signal Process.* 100 (2018) 482–500.
- [8] M. Wang, H. Shan, R. Lu, et al., Real-time path planning based on hybrid-VANET-enhanced transportation system, *IEEE Trans. Veh. Technol.* 64 (5) (2014) 1664–1678.
- [9] C.Y. Wang, P. Liu, H.Z. Qin, Review on intelligent path planning algorithm of mobile robots, *Transducer Microsyst. Technol.* 37 (8) (2018) 5–8.
- [10] B. Zhou, Y. Guo, N. Li, et al., Path planning of UAV using guided enhancement Q-learning algorithm, *Acta Aeronaut. Astronaut. Sinica* 42 (9) (2021) 498–505.
- [11] L. Claussmann, M. Revilloud, D. Gruyer, et al., A review of motion planning for highway autonomous driving, *IEEE Trans. Intell. Transp. Syst.* 21 (5) (2019) 1826–1848.
- [12] X.J. Xie, X.L. Qin, Q. Zhou, et al., A novel test-cost-sensitive attribute reduction approach using the binary bat algorithm, *Knowl.-Based Syst.* 186 (2019) 104938.
- [13] S.J. Liu, X.R. Tong, Urban transportation path planning based on reinforcement learning, *J. Comput. Appl.* 41 (1) (2021) 185–190.
- [14] Q. Zhou, Z.X. Sun, J.Y. Wu, et al., A location privacy preservation scheme based on consortium block-chain in VANET, *J. Nanjing Univ. Posts Telecommun. (Nat. Sci. Ed.)* 42 (6) (2022) 86–98.
- [15] Q. Zhou, T.L. Zhang, J.Y. Wu, et al., An adaptive path planning algorithm for local delivery of confidential documents based on block-chain, *J. Data Acquisit. Process.* 37 (6) (2022) 1314–1322.
- [16] X. Zhang, S. Xia, X. Li, et al., Multi-objective particle swarm optimization with multi-mode collaboration based on reinforcement learning for path planning of unmanned air vehicles, *Knowl.-Based Syst.* 250 (2022) 109075.
- [17] S.M. Sombolstan, A. Rasooli, S. Khodaygan, Optimal path-planning for mobile robots to find a hidden target in an unknown environment based on machine learning, *J. Ambient Intell. Humaniz. Comput.* 10 (2019) 1841–1850.
- [18] I. Konar, G. Chakraborty, S.J. Singh, et al., A deterministic improved Q-learning for path planning of a mobile robot, *IEEE Trans. Syst. Man Cybern.: Syst.* 43 (5) (2013) 1141–1153.
- [19] S. Džeroski, D.L. Raedt, K. Driessens, Relational reinforcement learning, *Mach. Learn.* 43 (2001) 7–52.

- [20] Y.L. Yuan, Z.L. Yu, Z.H. Gu, et al., A novel multi-step Q-learning method to improve data efficiency for deep reinforcement learning, *Knowl.-Based Syst.* 175 (2019) 107–117.
- [21] E.S. Low, P. Ong, C.Y. Low, R. Omar, Modified Q-learning with distance metric and virtual target on path planning of mobile robot, *Expert Syst. Appl.* 199 (2022) 117191.
- [22] L.G. Zhan, L.T. Sha, F. Xiao, et al., Automated windows domain penetration method based on reinforcement learning, *Chin. J. Netw. Inf. Secur.* 9 (4) (2023) 104–120.
- [23] Q.Y. Li, et al., INNES: An intelligent network penetration testing model based on deep reinforcement learning, *Appl. Intell.* 53 (22) (2023) 27110–27127.
- [24] X.S. Xu, J. Yuan, Path planning for mobile robot based on improved reinforcement learning algorithm, *J. Chin. Inertial Technol.* 27 (3) (2019) 314–320.
- [25] Y.Y. Zhang, C.H. Li, G.S. Zhang, et al., Local path planning of mobile robot based on improved Q-learning algorithm, *J. Shandong Univ. Technol. (Nat. Sci. Ed.)* 37 (2) (2021) 1–6.
- [26] L. Gao, T.L. Ma, K. Liu, et al., Application of improved Q-learning algorithm in path planning, *J. Jilin Univ. (Inf. Sci. Ed.)* 36 (4) (2018) 439–443.
- [27] L.J. Song, Z.Y. Zhou, Y.L. Li, et al., Research on path planning algorithm based on improved Q-learning algorithm, *J. Chin. Comput. Syst.* (2023) (online).
- [28] I. Syed, I. Moinul, U. Mohiuddin, Q-learning based particle swarm optimization algorithm for optimal path planning of swarm of mobile robots, in: *Proceedings of 2019 International Conference on Advances in Science, Engineering and Robotics Technology*, 2019, pp. 1–5.
- [29] Z. Bai, H. Pang, M. Liu, et al., An improved Q-learning algorithm and its application to the optimized path planning for unmanned ground robot with obstacle avoidance, in: *2022 6th CAA International Conference on Vehicular Control and Intelligence, CVCI*, 2022, pp. 1–6.
- [30] X. Wang, J. Liu, C. Nugent, et al., Mobile agent path planning under uncertain environment using reinforcement learning and probabilistic model checking, *Knowl.-Based Syst.* 264 (2023) 110355.
- [31] X. Tan, X.Q. Zhang, H.G. Shi, et al., Reinforcement-learning-based multi-slot rail empty wagon real-time distribution, *J. Transport. Eng. Inf.* 18 (4) (2020) 53–60.
- [32] R.S. Sutton, Learning to predict by the methods of temporal difference, *Mach. Learn.* 3 (1) (1988) 9–44.
- [33] E.S. Low, P. Ong, K.C. Cheah, Solving the optimal path planning of a mobile robot using improved Q-learning, *Robot. Auton. Syst.* 115 (2019) 143–161.
- [34] P.I. Aleksandr, K.S. Yakovlev, R. Suvorov, Grid path planning with deep reinforcement learning: Preliminary results, *Procedia Comput. Sci.* 123 (2018) 347–353.
- [35] T.G. Emmanouil, et al., A review of global path planning methods for occupancy grid maps regardless of obstacle density, *J. Intell. Robot. Syst.* 84 (2016) 829–858.
- [36] K. Sreelakshmy, et al., 3D path optimisation of unmanned aerial vehicles using q learning-controlled GWO-aoa, *Comput. Syst. Sci. Eng.* 3 (3) (2023) 45.
- [37] M. Tokic, Adaptive ϵ -greedy exploration in reinforcement learning based on value differences, in: *Annual Conference on Artificial Intelligence*, 2010, pp. 203–210.
- [38] S. Li, X. Xu, L. Zuo, Dynamic path planning of a mobile robot with improved Q-learning algorithm, in: *2015 IEEE International Conference on Information and Automation*, 2015, pp. 409–414.
- [39] C. Yan, X. Xiang, A path planning algorithm for UAV based on improved q-learning, in: *2018 2nd International Conference on Robotics and Automation Sciences, ICRAS*, 2018, pp. 1–5.
- [40] Y. Zhang, R. Ma, D. Zhao, et al., A novel energy management strategy based on dual reward function Q-learning for fuel cell hybrid electric vehicle, *IEEE Trans. Ind. Electron.* 69 (2) (2021) 1537–1547.
- [41] Z.Q. Li, Z. Pan, Y. Li, et al., Advanced root mean square propagation with the warm-up algorithm for fiber coupling, *Opt. Express* 31 (15) (2023) 23974–23989.
- [42] H. Iiduka, Appropriate learning rates of adaptive learning rate optimization algorithms for training deep neural networks, *IEEE Trans. Cybern.* 52 (12) (2021) 13250–13261.